

A Esteira da PIM-PF

Como uma nota econômica nasce sozinha, do dado ao PDF

Documento completo e didático do projeto — para quem nunca viu o repositório, não é técnico, e nem sabe o que é a PIM-PF.

Exemplo real deste documento: **abril 2026**

Gerado automaticamente em 11/06/2026 pela própria esteira.

Repositório: github.com/JoaoPauloZangrandi/pim-pf-nota

O que é este documento? Você está lendo um arquivo que o *próprio programa* montou. Ele explica, em ordem e em linguagem simples, tudo o que o projeto faz: o que é a pesquisa do IBGE, qual era o desafio, como o resultado é apresentado, como o computador faz tudo sozinho, por que (quase) nunca falha, e até mostra os pedaços mais importantes do código. Não é preciso saber programar para entender.

Como ler este documento

Leia na ordem, do começo ao fim — cada parte prepara a seguinte. Antes de tudo, quatro palavrinhas de programação que vão aparecer (explicadas como se você nunca tivesse visto código):

Quatro palavrinhas, sem susto

- **Programa / código:** uma lista de instruções que o computador segue, como uma receita bem detalhada.
- **Arquivo .py:** um pedaço do programa escrito na linguagem *Python*. Cada arquivo cuida de uma tarefa — como gavetas etiquetadas numa cozinha.
- **Função:** uma mini-receita dentro do arquivo que faz *uma* coisa específica (por exemplo, “buscar os dados”).
- **Erro tratado (exceção):** quando algo sai do esperado, o programa “levanta a mão” e avisa, em vez de seguir fingindo que está tudo bem. Esse cuidado é o coração deste projeto.

Sumário

1. O que é a PIM-PF (a pesquisa do IBGE)
2. O que os números significam (glossário)
3. O desafio da consultoria
4. A apresentação: o resultado de abril 2026
5. Como a esteira funciona, passo a passo
6. Onde está a automação
7. A rede de proteção (robustez)
8. A base teórica: Robert C. Martin, Cap. 7
9. O código por dentro (os trechos principais)
10. Como rodar e a estrutura do repositório

1. O que é a PIM-PF (a pesquisa do IBGE)

A **PIM-PF** significa *Pesquisa Industrial Mensal — Produção Física*. Em palavras simples: todo mês, o IBGE (o instituto oficial de estatísticas do Brasil) mede **o quanto as fábricas do país produziram** — de alimentos a automóveis, de petróleo a remédios.

É um dos primeiros sinais de como vai a economia: se a indústria produz mais, geralmente há mais empregos, mais vendas e mais investimento; se produz menos, é sinal de freada. Por isso bancos, consultorias e jornais aguardam esse número todo mês.

O IBGE divulga os dados num site chamado **SIDRA** por volta das **9h da manhã**, em datas marcadas num calendário. O número não vem em “toneladas”: vem como um **índice** (explicado no glossário a seguir).

2. O que os números significam (glossário)

O “índice” (base 2022 = 100)

A produção vira uma nota comparada com o ano de 2022, que vale 100. Um índice de 106 quer dizer “produzindo cerca de 6 por cento a mais do que a média de 2022”.

“Ajuste sazonal”

Alguns meses são naturalmente mais fracos ou fortes (dezembro tem festas; janeiro, férias). O ajuste sazonal *desconta* esse efeito do calendário, para podermos comparar um mês com o outro de forma justa.

As quatro leituras que a nota sempre traz

- **Variação no mês (a “margem”, $m/m-1$):** quanto mudou ante o mês anterior. É o termômetro do momento — mostra se a indústria está acelerando ou freando agora.
- **Variação no ano (“interanual”, $m/m-12$):** compara com o mesmo mês do ano passado (ex.: abril de 2026 contra abril de 2025). Mostra o quadro mais amplo.
- **Acumulado no ano:** soma o que foi produzido no ano até aqui, ante o mesmo período do ano anterior. Responde “como está o ano até agora?”.
- **Acumulado em 12 meses:** olha os últimos 12 meses juntos. É a medida mais “calma”, que mostra a tendência de fundo.

Como a indústria é dividida

- **Por tipo de produto (categorias econômicas):** bens de capital (máquinas e equipamentos = investimento), bens intermediários (insumos para outras indústrias) e bens de consumo — duráveis (carro, geladeira) e semi/não duráveis (comida, roupa).
- **Por setor (seções):** indústria *extrativa* (tirar da natureza: petróleo, minério) e *de transformação* (fabricar: alimentos, química, veículos). Juntas formam a “indústria geral”.

3. O desafio da consultoria

Imagine que você é *trainee* numa consultoria de economia. O IBGE divulga a PIM-PF por volta das 9h; a consultoria quer mandar aos clientes uma **nota informativa até as 10h**. O prazo é

curtíssimo. Fazer isso na mão todo mês seria arriscado: bastaria um detalhe para atrasar ou sair errado.

A solução é uma **esteira automatizada**: um programa que, sozinho, busca os dados, calcula, escreve o texto, desenha os gráficos e monta o PDF final — em segundos. E, como é coisa séria que vai para clientes, ele precisa ser **robusto**: nunca entregar um número errado em silêncio. Ou sai a nota certa, ou aparece um aviso claro do que deu errado.

4. A apresentação: o resultado de abril 2026

Esta é a informação que a esteira produziu para o mês de exemplo. (A nota enxuta de 1 página fica no arquivo `nota_pim_AAAA-MM.pdf`; aqui mostramos o mesmo conteúdo, explicado.)

Manchete: Produção industrial avançou 0,7% em abril 2026 (m/m-1, com ajuste sazonal).

Resumo: Interanual: +2,7% · Acumulado no ano: +1,7% · Em 12 meses: +0,7%.

Em abril 2026, a produção industrial brasileira avançou 0,7% ante o mês anterior na série com ajuste sazonal, com o índice dessazonalizado em 106,6 pontos (base 2022=100). Na comparação interanual, a indústria avançou 2,7%; no acumulado do ano, avançou 1,7%. O avanço mensal reforça a trajetória de alta também observada no acumulado em 12 meses.

As quatro leituras (indústria geral)

Leitura	Variação
Variação mensal (m/m-1, com ajuste sazonal)	+0,7%
Variação interanual (m/m-12)	+2,7%
Acumulado no ano	+1,7%
Acumulado em 12 meses	+0,7%

Por setor (seções)

Sector	m/m-1	Interanual	Ano	12 m
Indústrias extrativas	+3,1%	+10,6%	+9,3%	+7,3%
Indústrias de transformação	+0,3%	+1,2%	+0,3%	-0,5%

Por tipo de produto (grandes categorias econômicas)

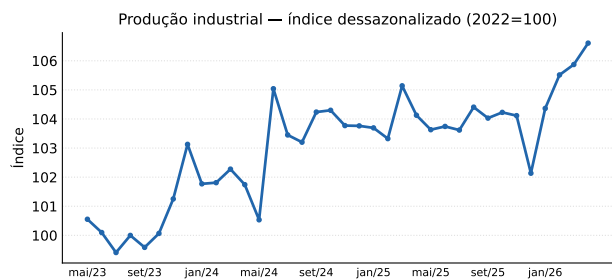
Categoria	m/m-1	Interanual	Ano	12 m
Bens de capital	+0,1%	-4,3%	-5,7%	-4,1%
Bens intermediários	+1,5%	+3,8%	+2,3%	+1,7%
Bens de consumo duráveis	-3,2%	-3,4%	+0,2%	-0,1%
Bens de consumo semiduráveis e não duráveis	-0,2%	+3,2%	+2,2%	-0,9%

Destaques entre atividades (comparação interanual)

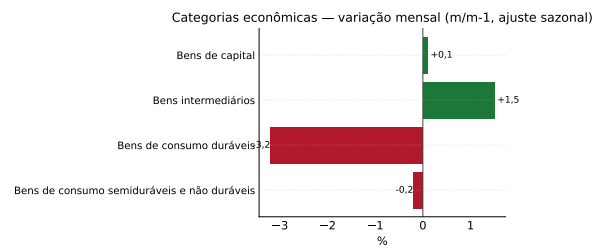
Maiores altas: Fabricação de coque, de produtos derivados do petróleo e de biocombustíveis (+13,3%); Fabricação de produtos de borracha e de material plástico (+3,8%); Fabricação de produtos alimentícios (+3,2%).

Maiores baixas: Fabricação de outros equipamentos de transporte, exceto veículos automotores (-7,9%); Fabricação de máquinas e equipamentos (-7,0%); Confeção de artigos do vestuário e acessórios (-6,5%).

Os gráficos



A linha mostra a produção ao longo do tempo.



Verde = alta no mês; vermelho = queda.

5. Como a esteira funciona, passo a passo

Pense numa **cozinha de restaurante** que prepara o mesmo prato toda manhã. Eis o que acontece, em ordem, quando o programa roda:

1. **Alguém aperta o “play”.** Você (ou um robô agendado) dá um comando. Daí em diante o programa toca sozinho. (*arquivo: a porta de entrada do programa.*)
2. **O chefe de cozinha comanda.** Um arquivo central chama cada ajudante na ordem certa e já entra com um plano B caso algo falhe. (*arquivo: `pipeline.py`.*)
3. **Buscar os ingredientes no fornecedor (o IBGE).** O programa acessa o site de dados do IBGE pela internet e baixa os números. (*arquivo: `sidra_client.py`.*)
4. **Conferir se o “prato do dia” já saiu.** Se o mês pedido ainda não foi divulgado, ele avisa e para — não serve dado velho como novo.
5. **Pegar os três ingredientes:** o resultado do mês, o histórico de 3 anos (para o gráfico) e os números por tipo de produto.
6. **Guardar uma cópia “na geladeira”.** Salva os dados crus num arquivo. Se amanhã o site estiver fora do ar, dá para usar essa cópia. (*arquivo: `cache.py`.*)
7. **Conferir se os ingredientes vieram certos.** Verifica se os dados têm o formato esperado; se o IBGE mudar o layout, para na hora. (*arquivo: `schema.py`.*)
8. **Transformar o cru no prato.** Calcula as variações, separa por setor e acha as maiores altas e baixas. (*arquivo: `transform.py`.*)
9. **Escrever o texto.** Monta a manchete e o contexto por regras fixas — nunca “inventa”. (*arquivo: `analysis.py`.*)
10. **Desenhar os gráficos.** (*arquivo: `charts.py`.*)
11. **Montar o PDF bonito.** Preenche um molde em LaTeX e “imprime” o PDF. (*arquivo: `render_latex.py`.*)
12. **Entregar e dar o veredito.** Gera os documentos e avisa: deu certo, o mês não saiu, ou houve um erro (com explicação).

6. Onde está a automação

Há dois níveis:

- **Um comando faz tudo.** Rodar `python -m pim_report --periodo last` executa toda a esteira (buscar, tratar, analisar, gerar PDF e HTML) sem nenhum passo manual.
- **Sem ninguém digitar nada.** O projeto tem um “robô” no GitHub (um *workflow* agendado) que dispara perto do horário da divulgação do IBGE, roda a esteira, guarda os arquivos e — se algo falhar — **abre um chamado (issue) automaticamente**, para o time agir antes das 10h. Há também um robô que, a cada mudança no código, roda os testes e a checagem de qualidade.

7. A rede de proteção (robustez)

“Minimizar o risco de chegar no escritório e o código não ter funcionado” é o critério central. Cada linha abaixo é uma trava de segurança:

Trava de segurança	O que evita	Onde
Tempo-limite e novas tentativas	Travar para sempre ou desistir por uma queda passageira de rede	<code>sidra_client.py</code>
Tradução de erros	Que detalhes técnicos da internet contaminem o resto do programa	<code>sidra_client.py</code>
Conferência do formato	Gerar números errados se o IBGE mudar o layout	<code>schema.py</code>
“O mês já saiu?”	Mandar nota com mês velho fingindo ser novo	<code>pipeline.py</code>
Cópia na geladeira (cache)	Ficar sem nota quando o site cai (usa a cópia e avisa)	<code>cache.py</code>
Plano B na impressão	Falhar só porque uma ferramenta de PDF não roda	<code>render_latex.py</code>
Testes automáticos	Quebrar sem ninguém perceber antes	<code>tests/</code>
Robô que abre chamado	Descobrir o problema só quando o cliente reclama	GitHub Actions

A promessa final: em qualquer execução, ou sai um **PDF correto**, ou aparece um **erro claro com explicação e registro** — *nunca* um documento errado disfarçado de certo.

8. A base teórica: Robert C. Martin — *Código Limpo*, Cap. 7

O capítulo 7 do livro *Código Limpo* trata de “Tratamento de Erro”: como um programa deve lidar com imprevistos sem virar uma bagunça. O projeto segue os 8 princípios; cada um foi aplicado num ponto concreto do código:

#	Princípio	Onde aplicamos
1	Use exceções, não códigos de retorno	sidra_client.py, transform.py, render_latex.py
2	Escreva try/except/finally primeiro (TDD)	tests/test_sidra_client.py, tests/test_render.py, pipeline.py
3	Exceções não verificadas + hierarquia de domínio	exceptions.py
4	Forneça contexto nas exceções	exceptions.py, sidra_client.py, render_latex.py
5	Defina classes segundo a necessidade do chamador	sidra_client.py, exceptions.py
6	Defina o fluxo normal (Special Case pattern)	special_cases.py, pipeline.py
7	Não retorne None	cache.py, transform.py, sidra_client.py
8	Não passe None	exceptions.py (guard) + todos os módulos

9. O código por dentro (os trechos principais)

A seguir, os pedaços mais importantes do código (comentários simplificados). Não precisa entender cada linha — repare na *ideia* de cada um.

9.1 A regra de ouro: proibir o “nada” (None) na entrada

Toda função confere se recebeu o que precisava; se veio vazio, avisa na hora.

```
def exigir_nao_nulo(valor, nome, *, operacao=None):
    if valor is None:
        raise InvalidParameterError(
            f"Parametro obrigatorio ausente: {nome} nao pode ser None.",
            contexto={"parametro": nome, "operacao": operacao})
    return valor
```

9.2 O wrapper que conversa com o IBGE e traduz qualquer erro

Esta é a peça que “liga para o fornecedor”. Qualquer problema vira um aviso claro e padronizado.

```
try:
    resposta = self._session.get(url, timeout=self.timeout)
except (requests.Timeout, requests.ConnectionError) as erro:
    raise SidraConnectionError("Falha de conexao/timeout no SIDRA",
```

```

        contexto={"url": url}) from erro

if resposta.status_code >= 400:
    raise SidraHTTPError(f"SIDRA respondeu HTTP {resposta.status_code}",
                        status_code=resposta.status_code)
try:
    dados = resposta.json()
except ValueError as erro:
    # veio algo que nao e JSON
    raise SidraPayloadError("Resposta do SIDRA nao e JSON valido") from erro
if len(dados) <= 1:
    # so o cabecalho => sem dados
    raise SidraEmptyError("Consulta sem registros de dados")
return dados

```

9.3 Tentar de novo, mas só quando faz sentido

Se a falha foi passageira (rede instável, servidor ocupado), tenta de novo, esperando um pouco mais a cada vez. Se foi erro definitivo, nem insiste.

```

def _e_transitorio(exc):
    if isinstance(exc, SidraConnectionError): return True
    if isinstance(exc, SidraHTTPError):      return exc.transitorio # 5xx
    return False

Retryng(stop=stop_after_attempt(self.max_tentativas),
        wait=wait_exponential(multiplier=self.espera_inicial),
        retry=retry_if_exception(_e_transitorio), reraise=True)

```

9.4 O “caso especial”: mês ainda não publicado

Em vez de um erro confuso, devolve um objeto que diz claramente: este mês ainda não saiu.

```

class PeríodoNaoPublicado:
    publicado = False # nunca se diz publicado
    def mensagem(self):
        return (f"PIM-PF do período {self.período_solicitado} ainda nao "
                f"publicada (mais recente: {self.período_disponível}).")

```

9.5 A geladeira: usar a cópia quando o site cai

```

def carregar_mais_recente(dir_cache):
    arquivos = sorted(dir_cache.glob("*.json"))
    if not arquivos:
        raise CacheVazioError("Nenhum cache disponível") # nao retorna None
    registros = json.loads(arquivos[-1].read_text(encoding="utf-8"))
    return CacheBruto(registros, _data_do_nome(arquivos[-1]), arquivos[-1])

```

9.6 Imprimir o PDF com plano B

Tenta a primeira ferramenta; se falhar, tenta a segunda. Se nenhuma der, guarda o registro do erro e avisa.

```

for cmd in _comandos_candidatos(destino, caminho_tex): # latexmk, depois pdflatex

```

```

    resultado = executar(cmd, cwd=str(destino))
    if resultado.returncode == 0 and caminho_pdf.exists():
        return caminho_pdf
raise LatexCompilationError("Falha ao compilar o LaTeX",
                             log_path=str(caminho_log))

```

9.7 O maestro: a sequência inteira, com plano B embutido

Repare na estrutura “tente / se falhar / no fim sempre”. É o desenho que garante a promessa final.

```

try:
    try:
        pacote = coletar_pacote(client, alvo)    # busca ao vivo
        salvar_bruto(pacote, ...)                # guarda a copia
    except SidraError as erro:                  # o site caiu?
        cache = carregar_mais_recente(...)      # usa a copia da geladeira
        pacote, de_cache = cache.registros, True
    dados      = construir_dados(pacote, ...)
    narrativa = gerar_narrativa(dados)
    pdf  = renderizar_nota(dados, narrativa, graficos, ...)
    html = renderizar_relatorio(dados, narrativa, graficos, ...)
except PimReportError as erro:
    logger.error("Esteira falhou: %s", erro); raise
finally:
    logger.info("=== Fim da esteira ===")

```

10. Como rodar e a estrutura do repositório

Para rodar no computador

```
# 1) preparar o ambiente (uma vez)
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # Windows
pip install -e ".[dev]"

# 2) gerar a nota do mes mais recente
python -m pim_report --periodo last

# 3) (opcional) refazer um mes especifico
python -m pim_report --periodo 202504
```

Os arquivos saem na pasta `output/`: a nota em PDF, este manual, o `.tex` preenchido, o `relatorio.html` e os gráficos.

O que cada arquivo do código faz

Arquivo	Responsabilidade
<code>config.py</code>	Centraliza códigos de tabela/variáveis/classificações do SIDRA, caminhos e parâmetros de rede. Troca de código = uma linha.
<code>exceptions.py</code>	Hierarquia de exceções de domínio (raiz <code>PimReportError</code>) + guard <code>exigir_nao_nulo</code> . Princípios 1, 3, 4, 5, 8.
<code>special_cases.py</code>	<code>PeriodoNaoPublicado</code> (Special Case de Fowler). Princípios 6 e 7.
<code>sidra_client.py</code>	Wrapper da API: timeout, retry exponencial (tenacity) só em transitórios, e tradução de erros para <code>SidraError</code> . Princípio 5.
<code>schema.py</code>	Validação pandera do <code>DataFrame</code> cru: falha cedo e clara se o IBGE mudar o layout.
<code>cache.py</code>	Persiste o JSON cru e habilita o modo de degradação (usa o cache mais recente).
<code>transform.py</code>	As 4 leituras, categorias econômicas, seções e destaques + série dessaz.
<code>analysis.py</code>	Narrativa determinística (manchete, contexto, destaques) derivada dos dados.
<code>charts.py</code>	Figuras em PDF (LaTeX) e PNG (HTML), estilo limpo.
<code>render_latex.py</code>	Jinja2 -> <code>.tex</code> -> <code>latexmk/pdflatex</code> -> PDF (nota + manual), com tratamento de falha.
<code>render_html.py</code>	Relatório HTML explicativo, autossuficiente.
<code>pipeline.py</code>	Orquestra tudo com <code>try/except/finally</code> no topo e logging por etapa.

Para saber mais (fontes oficiais)

- PIM-PF — pagina oficial (metodologia e calendario)

<https://www.ibge.gov.br/estatisticas/economicas/industria/9294-pesquisa-industrial-mensal-producao-fisica-b>

- SIDRA — PIM-PF Brasil
<https://sidra.ibge.gov.br/home/pimpfbr/brasil>
- API SIDRA
<https://apisidra.ibge.gov.br/>
- Exemplo de narrativa (materia do IBGE)
<https://agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/27576-pandemia-faz>
- Metadados oficiais — tabela 8888
<https://servicodados.ibge.gov.br/api/v3/agregados/8888/metadados>
- Repositorio do projeto (GitHub)
<https://github.com/JoaoPauloZangrandi/pim-pf-nota>
- Robert C. Martin (2011). *Código Limpo* — Capítulo 7: Tratamento de Erro.